

AI 시스템반도체 SW개발자(2기)

프로젝트 결과보고서

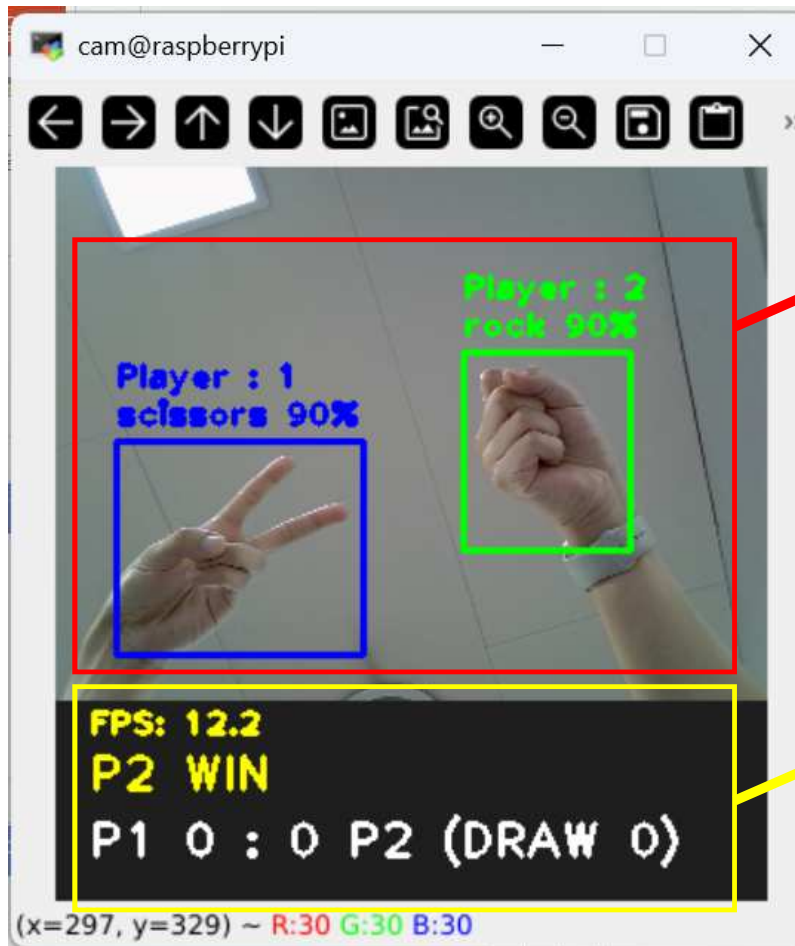
RPS with YOLO

이시원

목차구성

1. 결과 요약
2. 개발 목표 및 개발 단계
3. 단계별 구현 방식
4. 결과 분석
5. 향후 발전 가능성

1. 결과 요약



상대적 좌표 기반 플레이어 구분
판정 결과 및 확률 표시

승자 판단 및 스코어 보드

2. 개발 단계

단계	구현 목표
1	상대적 좌표를 이용한 플레이어 구분
2	플레이어 간 승패 구분
3	2명의 플레이어가 아닌 경우 예외 처리
4	카운트 다운 이후 결과 판정 및 프레임 고정
5	스코어 보드 및 기능키 구현

3. 단계별 구현 방식

1. 상대적 좌표를 이용한 플레이어 구분

```
def processImage(frame):  
  
    # 최종 BB만 그리기  
    draw_boxes = [(boxes[i], scores[i], classes[i]) for i in keep]
```

```
    boxes_pixel = []  
    for (x,y,w,h), sc, cid in draw_boxes:  
        x *= IMG_SIZE; y *= IMG_SIZE  
        w *= IMG_SIZE; h *= IMG_SIZE
```

```
        x1 = (x - pad_x) / r  
        y1 = (y - pad_y) / r  
        x2 = (x + w - pad_x) / r  
        y2 = (y + h - pad_y) / r
```

```
        x1 = int(np.clip(x1,0,frame.shape[1]))  
        y1 = int(np.clip(y1,0,frame.shape[0]))  
        x2 = int(np.clip(x2,0,frame.shape[1]))  
        y2 = int(np.clip(y2,0,frame.shape[0]))
```

```
        # 프로젝트 1. 좌/우 플레이어 구분  
        # 좌표를 비교해서 P1(왼쪽), P2(오른쪽)  
        boxes_pixel.append([x1,y1,x2,y2,sc,cid])
```

```
    boxes_pixel.sort(key= lambda x :x[0])
```

Boxes_pixel 예시

	x1	Y1	X2	Y2	Sc	cid
1행	9	80	140	190	0.919	0
2행	148	54	309	230	0.888	2

```
    for i, (x1,y1,x2,y2,sc,cid) in enumerate(boxes_pixel):
```

```
        player = (1 if i == 0 else 2) if len(boxes_pixel) == 2 else 1
```

```
        # BB 표시
```

```
        cv2.rectangle(frame, (x1, y1), (x2, y2), colorList[cid], 2)
```

```
        # 판정 결과 표시
```

```
        cv2.putText(frame, f"Player : {player}", (x1,y1-24), cv2.FONT_HERSHEY_PLAIN, 1, colorList[cid], 2)
```

```
        cv2.putText(frame, f'{{ansToText[cid]}} {{int(sc*100)}}%', (x1,y1-7), cv2.FONT_HERSHEY_PLAIN, 1, colorList[cid], 2)
```

3. 단계별 구현 방식

2. 승패 구분

```
# 프로젝트3 : 손이 2개가 아닐 때 예외처리
if len(keep) != 2:
    round_result = '2 HANDS ARE NEEDED'
else:
    # 프로젝트2 : 승자 찾기
    # {0:'scissors', 1:'rock', 2:'paper'}
    p1 = boxes_pixel[0][5] # class id
    p2 = boxes_pixel[1][5]

    if p1 == p2:
        round_result = 'DRAW'
    elif p2 == ((p1+2) % 3) :
        round_result = 'P1 WIN'
    else:
        round_result = 'P2 WIN'
```

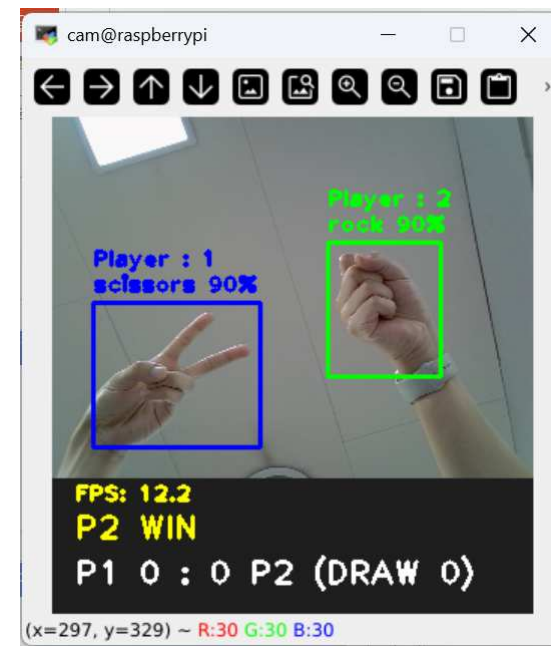
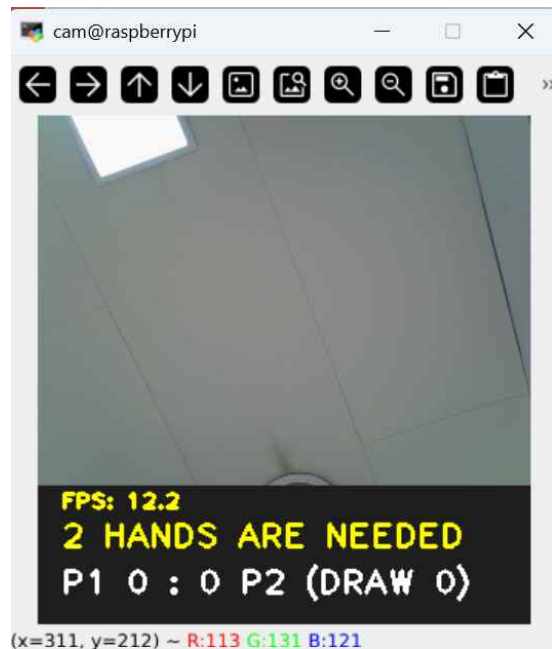
	P1 가위 (0)	P1 바위 (1)	P1 보 (2)
P2 가위 (0)	DRAW	P1 승	P2 승
P2 바위 (1)	P2 승	DRAW	P1 승
P2 보 (2)	P1 승	P2 승	DRAW

표를 그려 봤을 때,
P1은 P2와 class id가 2 차이날 때
승리하는 조건을 발견함

3. 단계별 구현 방식

3. 예외 처리

```
# 프로젝트3 : 손이 2개가 아닐 때 예외처리  
if len(keep) != 2:  
    round_result = '2 HANDS ARE NEEDED'
```



3. 단계별 구현 방식

4. 카운트 다운 이후 결과 판정 및 프레임 고정

```
startTime = time.time()
```

```
# 게임 상태 관리
```

```
state = 'LIVE'
```

```
countdown_start = None
```

```
frozen_frame = None
```

```
frozen_result = None
```

```
while(cap.isOpened()):
```

```
    if state == 'FROZEN':
```

```
        frame = frozen_frame.copy()
```

```
        round_result = frozen_result
```

```
    else:
```

```
        ret, frame = cap.read() # 사진 찍기 -> (240, 320, 3)
```

```
        if not ret: break
```

```
        # 이미지 처리
```

```
        round_result = processImage(frame)
```

```
# 프로젝트 4 : 3,2,1 카운트 다운 후 판정 순간 프
```

```
elif key == ord('r') and state == 'LIVE':
```

```
    state = 'COUNTDOWN'
```

```
    countdown_start = time.time()
```

```
# 프로젝트 5 : 재시작 키 추가
```

```
elif key == ord(' ') and state == 'FROZEN':
```

```
    state = 'LIVE'
```

```
    frozen_frame = None
```

3. 단계별 구현 방식

4. 카운트 다운 이후 결과 판정 및 프레임 고정

```
if state == 'COUNTDOWN':  
    elapsed = time.time() - countdown_start  
    time_remaining = 3 - int(elapsed)  
  
    if time_remaining > 0:  
        cv2.putText(frame, str(time_remaining), (140, 140), cv2.FONT_HERSHEY_PLAIN, 6, (0,255,255), 4)  
    else:  
        # 카운트다운 종료 -> 이 순간의 frame(판정 결과 포함)을 고정  
        frozen_frame = frame.copy()  
        state = 'FROZEN'  
        frozen_result = round_result  
        if frozen_result in score:  
            score[frozen_result] += 1
```

3. 단계별 구현 방식

5. 스코어 보드 및 기능키 구현

이미지 영역 아래에 Padding을 이용해
스코어 보드 영역 구현

```
# ---- 하단 패딩 영역 추가 ----
```

```
canvas = cv2.copyMakeBorder(frame, 0, PAD_HEIGHT, 0, 0, cv2.BORDER_CONSTANT, value=(30, 30, 30))
```

```
cv2.putText(canvas, round_result, (10, frame.shape[0] + 25), cv2.FONT_HERSHEY_PLAIN, 1.5, (0, 255, 255), 2)
```

```
scoreboard_text = f"P1 {score['P1 WIN']} : {score['P2 WIN']} P2 (DRAW {score['DRAW']})"
```

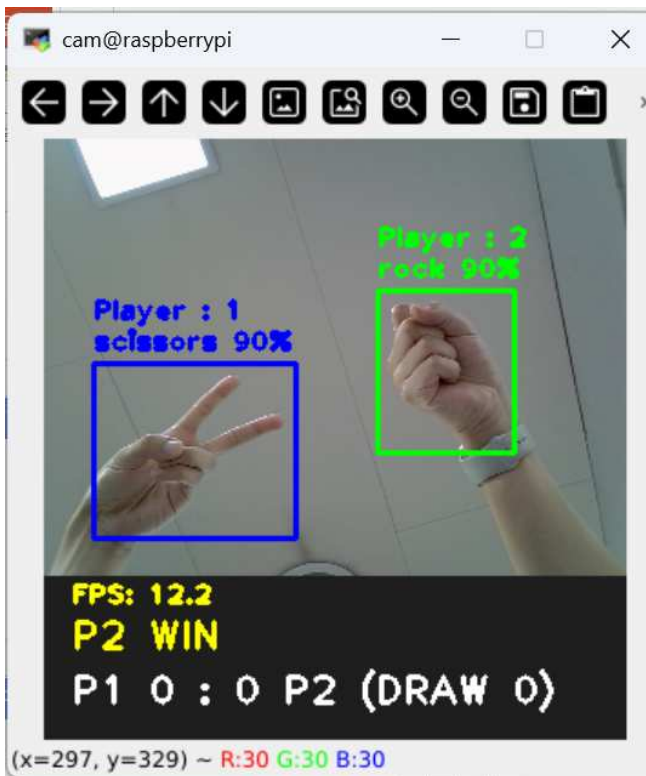
```
cv2.putText(canvas, scoreboard_text, (10, frame.shape[0] + 55), cv2.FONT_HERSHEY_PLAIN, 1.5, (255, 255, 255), 2)
```

```
# 이미지 출력
```

```
cv2.imshow('cam', canvas)
```

4. 결과 분석

1. 플레이어 별 RPS 구분과 판정 확률 확인 가능
2. 스코어 보드에서 승자와 현재까지의 점수 및 무승부 횟수 확인 가능



5. 향후 발전 가능성

1. 현재 모델의 한계점

- 프레임 고정시, 사용자가 갑자기 바꾸는 경우 사진이 흔들리면서 정확한 판정이 불가함

2. 개선 방안

- Scissors, Rock, Paper 를 제외한 Undetected 클래스를 만들고 데이터를 마련한 뒤 새로 훈련 시킴

3. 기대 효과

- 사진이 흔들려서 RPS 로 판정이 불가할 경우 Undetected 판정, 승자 판단 보류